Phase 2: Requirements Engineering

The four roles use one program. It runs on a school computer or the local network and keeps its data in SQLite, so it works without internet.
This phase has three parts:

1. Stakeholder analysis and solving conflicts
2. The Software Requirements Specification (SRS)
3. The use case model

1. Stakeholder Analysis

The stakeholders

Stakeholder	What they want
Principal	A simple system. Marks must be checked by the HoD before students see them.
Lecturer	To make assignments, set deadlines, mark work, give feedback, and give extra time to students who have good reasons.
Head of Department (HoD)	To check marks before they are released. To make sure the late-work rules are followed.
Student representative	To submit work and fix mistakes before the deadline. To get written feedback. To have late work treated fairly.
System administrator / IT officer	To manage users and school data. To keep passwords safe. To make sure the system works on old computers with no internet.
School management	A system that works well, helps school work, and keeps student information safe.
Project developer	Clear, agreed requirements so the system can be designed, built and tested correctly.

Conflicts and how we solve them

The first statements from stakeholders disagree in some places. Some details are also missing. Below are the decisions we propose.

Conflict 1: When can students see their marks?

- The Principal says students must not see marks until the HoD has checked them.
- The lecturer wants students to see marks right after marking.

Solution: The lecturer marks the work and writes feedback. The marks go to the HoD. Students see the marks only after the HoD approves them. This protects the school's checking (moderation) process.

Conflict 2: Extra time (extensions)

- The lecturer wants to give extra time to students with good reasons.

- Nobody said how to record extensions or how they change late penalties.

Solution: The lecturer can give and record an extension for a student. The system uses the new deadline when it checks if work is late. The school must confirm if proof or approval from another person is needed first.

Conflict 3: Late work

- The HoD says late work loses 10% of the total mark for each late day, for up to three days. After that, the mark is zero.
- The student representative says late work should not just be refused.

Solution: The system accepts late work for up to three days after the deadline and takes off the penalty. Work more than three days late gets zero. The system does not quietly reject it. The school must confirm how to count a late day, including part of a day.

Conflict 4: Fixing and resubmitting work

- Students want to fix mistakes and submit again before the deadline.
- Nobody said how many times they can resubmit, or which version the lecturer marks.

Solution: Students can resubmit before the deadline. The system keeps the details of every submission and clearly shows the latest valid one. The school must confirm if there is a limit on resubmissions.

Conflict 5: Written feedback

- Students want written feedback, not just a mark.
- Nobody said if feedback is needed for every assignment.

Solution: Lecturers can save written feedback with the marks. Students see the feedback and marks only after the HoD approves them.

Conflict 6: Working offline and working fast

- The IT officer says the school has old computers and bad internet.
- The system must work offline, keep passwords safe, and be fast with 800 students.

Solution: The system uses a local SQLite database. Its main functions must not need internet. Passwords must never be saved as plain text. We must test speed with data for 800 students on the school's own computers.

Conflict 7: Easy to use

- The Principal says teachers are not IT experts.
- Confusing menus or long steps will not work for them.

Solution: The system uses simple menus, clear messages, and options that fit each role. User instructions will be given when the system is installed.

2. Software Requirements Specification (SRS)

Scope

The LMS will support:

- Making assignments and submitting work
- Marking and written feedback
- Extensions and late penalties
- Checking (moderation) of marks

It has four roles: students, lecturers, HoDs and administrators. It uses C++17 and SQLite and works offline.

Not included in this project:

- Online payments
- Video calls
- Automatic marking
- Plagiarism checking
- Web and mobile apps

Users

- **Students:** see assignments, submit and resubmit work, and see released marks and feedback.
- **Lecturers:** make assignments, set deadlines, give extensions, mark work and write feedback.
- **Heads of Department:** check marks and approve their release to students.
- **System administrators:** manage user accounts and the school data the LMS needs.

Priority: MoSCoW

MoSCoW is a way to decide which requirements matter most.

- **Must:** The system needs this to do its main job.
- **Should:** This is important, but the system can work for a short time without it.
- **Could:** This is nice to have if there is time.
- **Won't have:** This is not in this version.

The priorities below are our proposal.

Functional requirements

Functional requirements say **what the system must do**.

ID	Requirement	Priority	How we check it
FR-01	The system lets an allowed user log in with their account details.	Must	Test with correct and wrong login details.
FR-02	The system limits each user to what their role allows.	Must	Test allowed and not-allowed actions for all four roles.
FR-03	The administrator can create and manage user accounts.	Must	Create, view, change and turn off a test account.
FR-04	The administrator can manage courses, departments and student enrolments.	Must	Add and change test records. Check they are saved.
FR-05	Lecturers can create assignments with a title, instructions, deadline and total mark.	Must	Create an assignment. Check the saved details.

ID	Requirement	Priority	How we check it
FR-06	Students can see the assignments for their courses.	Must	Check a student sees the right assignments.
FR-07	Students can submit work for an assignment.	Must	Submit work. Check the system records it.
FR-08	Students can fix and resubmit work before the deadline.	Must	Submit a first and second version before the deadline.
FR-09	Lecturers can record an extension for a student.	Must	Record an extension. Check the new deadline is used.
FR-10	The system works out late penalties using the correct deadline and the late-work rules.	Must	Test on-time work, work 1 to 3 days late, and work more than 3 days late.
FR-11	Lecturers can record marks and written feedback.	Must	Save a mark and feedback. Check both are stored.
FR-12	The HoD can review marks and approve their release before students see them.	Must	Check hidden marks stay hidden and approved marks show.
FR-13	The system records every mark change: who, when, old mark, new mark and reason.	Must	Change a mark. Look at the audit record.
FR-14	Allowed users can see the information they need for their work.	Should	Check each role sees the right records and nothing restricted.

Non-functional requirements

Non-functional requirements say **how well the system must work**. They cover speed, safety, ease of use and reliability.

These targets are proposals. We must check them on the school's computers before we approve them.

ID	Requirement	Priority	How we check it
NFR-01	A login takes 2 seconds or less on the school computer with 800 students in the database.	Should	Time many logins under these conditions.

ID	Requirement	Priority	How we check it
NFR-02	A normal database action, such as opening an assignment list, takes 2 seconds or less with 800 students.	Should	Time actions using a test database with 800 students.
NFR-03	The main functions work with no internet.	Must	Turn off internet. Test login, assignments, submission and marking.
NFR-04	Passwords are never saved as plain text. They are saved as salted hashes made by a trusted password library.	Must	Look in the database. Check the passwords are salted hashes.
NFR-05	A user cannot see or change information their role does not allow.	Must	Try forbidden actions with each of the four roles.
NFR-06	The system shows a clear error message for bad input or database problems. It must not crash in the planned error tests.	Must	Test bad input and pretend database errors. Check what the program does.
NFR-07	The system shows clear menus and instructions for the main tasks of each role.	Should	Ask one test user from each role to do a main task. Write down any problems.
NFR-08	Every mark change has an audit record: user, time, old mark, new mark and reason.	Must	Make several mark changes. Check every one has a full record.

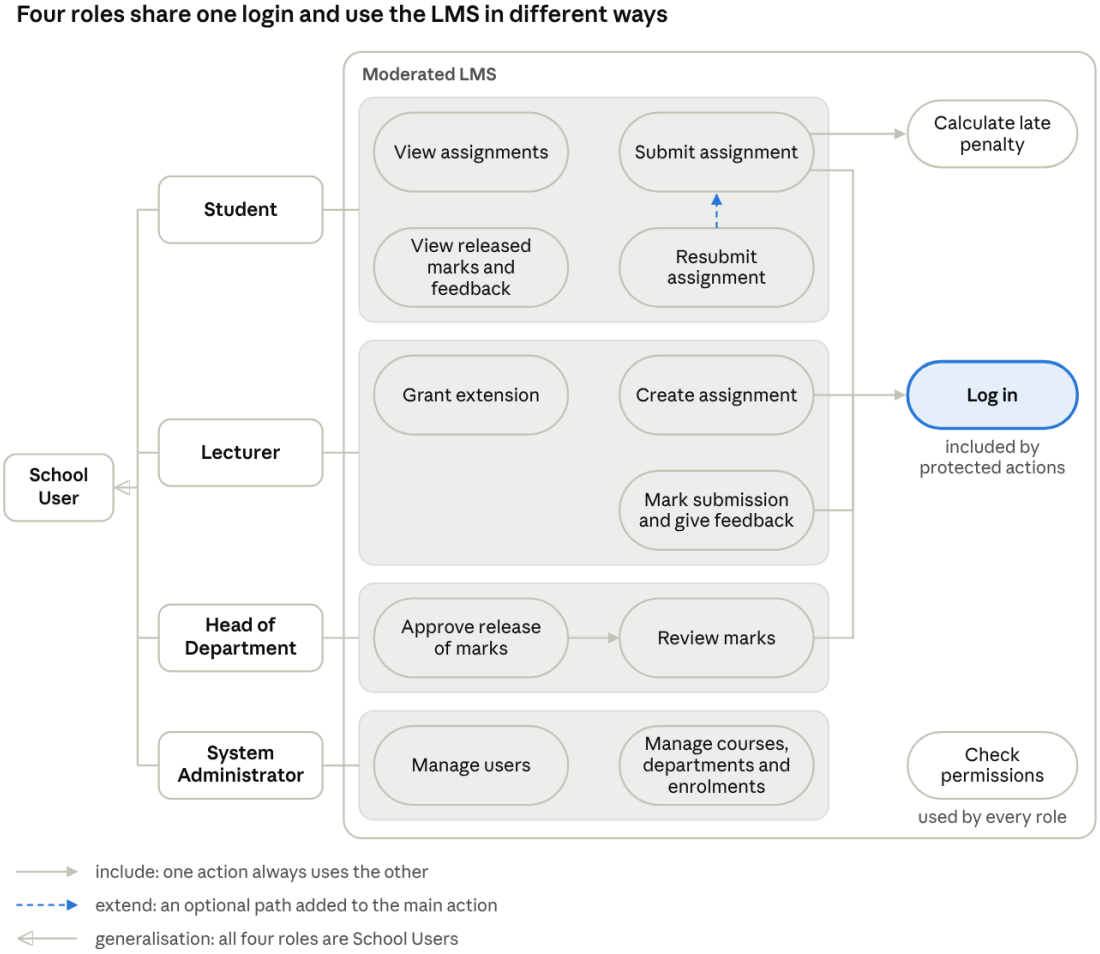
3. Use Case Model

A use case describes something a user wants to do with the system.

Actors

An actor is a user of the system. The diagram has four actors:

1. **Student:** sees assignments, submits and resubmits work, and sees released marks and feedback.
2. **Lecturer:** creates assignments, gives extensions, marks work and writes feedback.
3. **Head of Department:** reviews marks and approves their release.
4. **System Administrator:** manages user accounts, departments, courses and enrolments.



Four actions include Log in. Submit assignment also includes the late penalty check, and Resubmit extends Submit.

Fully Dressed Use Cases

Use Case 1: Submit Assignment

Main actor: Student
Goal: Submit work for an assignment.
Before it starts:

- The student has an account and is logged in.
- The student is enrolled in the course.
- The assignment exists and is open for submission.

Trigger: The student picks an assignment and chooses to submit work.
Main steps:

1. The student opens the assignment list.
2. The system shows the student's assignments.
3. The student picks an assignment.
4. The system shows the instructions and the deadline.

5. The student selects or enters the work.
6. The student confirms the submission.
7. The system checks the submission is valid.
8. The system checks the deadline, including any extension.
9. The system saves the submission and the time.
10. If the work is late, the system applies the late-work rule.
11. The system shows a confirmation message.

Use Case 2: Create Assignment

Main actor: Lecturer
Goal: Create an assignment for a course.
Before it starts:

- The lecturer has an account and is logged in.
- The lecturer is allowed to manage the course.

Trigger: The lecturer chooses to create an assignment.
Main steps:

1. The lecturer opens the course.
2. The lecturer selects "Create Assignment."
3. The system shows the details to enter.
4. The lecturer enters the title and instructions.
5. The lecturer enters the deadline and total mark.
6. The lecturer checks the information.
7. The lecturer confirms.
8. The system checks the information.
9. The system saves the assignment.
10. The system shows a confirmation message.

Use Case 3: Review and Release Marks

Main actor: Head of Department
Goal: Review marks and approve their release to students.
Before it starts:

- The HoD is logged in.
- The HoD is allowed to review marks for the department.
- A lecturer has entered marks for the submissions.

Trigger: The HoD opens the list of marks waiting for review.
Main steps:

1. The HoD opens the department or course.
2. The system shows marks waiting for review.
3. The HoD picks a set of marks or one submission.
4. The system shows the mark and related information.
5. The HoD checks the information.
6. If it is correct, the HoD approves the release.
7. The system records the approval.
8. The system changes the mark's status to released.

9. The system lets the student see the mark and feedback.
10. The system shows a confirmation message.